IT UNIVERSITY OF COPENHAGEN

Counter-strike 2 Game data collection with cheat labelling

19/05/25

Mille Mei Zhen Loo
milo@itu.dk

Gert Lužkov
gelu@itu.dk

Supervisor

Paolo Burelli
pabu@itu.dk

STADS code : KIREPRO1PE
Examination group : V24KIREPRO1PE029

IT University of Copenhagen
M.Sc. Computer Science
Research project

Abstract

In 2023 Valve launched Counter-Strike 2(CS2). Ever since the release of CS2 the game had problems with users deploying cheats to gain unfair advantages. It is known that machine learning can be used to classify cheaters, however this is a very data intensive task, and no dataset with cheater annotations exists for CS2. This paper aims to build a dataset with cheater labelling for the video game Counter-Strike 2. This was achieved by web scraping the CS2 match sharing site csstats.gg for match sharing codes and player data. This yielded 11 088 match sharing codes stored in a csv file format. The match sharing codes representing competitive and premier type matches had a ratio of games without any VAC-banned players to games with at least one VAC-banned player of 30 to 1. Due to this bias towards data without VAC-banned players, a subset of matches with no VAC-banned players was chosen with a focus of keeping the representation of maps equal. This yielded 850 demo files with the distribution of 350 containing at least one VAC-banned user(cheater), and 500 where users present had not been banned within 2 days of the match taking place. The quality of the data labels was determined by manual review of 50 demo files where a user who was later banned was present, and 50 where no banned users were present. The manual review resulted in "cheater" label having a precision of 92.6% and the "not cheater" label having a precision of 97.2%. Conclusively, webscraping the match sharing service csstats.gg proved to be an effective method for gathering data. However, caution is advised if such data is to be used in training a machine learning model as outliers occur.

Contents

1 Glossary

Aim hack A class of cheats involving the use of one or multiple aim and/or trigger assisting cheats. These can include but are not limited to recoil control system, trigger aim and trigger bot. 4, 18

Anti-aim A technique often used in cheats that contorts the playable character in a manner, such that it's hitbox is hard or impossible to target. If cheats are involved, they are often very easily identified due to unnatural movement. 2, 4

Bunny hopping A movement style where a player continuously jumps while moving and upon landing instantly jumps again within the same server tick. Counter-Strike 2 (CS2) uses a friction variable to slow the player down after landing, but if another jump is made upon the same server tick as landing, the player effectively removes all friction. This allows for increased movement speeds. 1, 4, 11, 18

CAPTCHA Completely Automated Public Turing test to tell Computers and Humans Apart[1]. 8, 9

CS Counter-Strike. 3, 6–8, 11, 20

CS2 Counter-Strike 2. 1, 3–5, 7, 10, 11, 15, 19, 20, 24

CS:GO Counter-Strike: Global Offensive. 1, 4, 7, 10

CT counter-terrorists. 3

Demo file A recording of a CS2 match. Every single event is recorded in these files in the form of server ticks. 1, 5–11, 14, 15, 17, 20, 27

Faceit A third party match provider for CS2 with their own elo system and anti-cheat software. 8, 10, 19

FPS first-person shooter. 3, 4, 6

Match sharing code A code that allows users to download a game stored on the Valve/Steam server. Games are only available on the servers for 30 days, after which the game is inaccessible. 1, 7–10, 13, 14, 20

Match sharing service A service provided by a third party, where users can upload their matches. The services often provide the means of analysing play styles with the intention of improving performance for the users. 1, 7, 20

Mod modification. 3

Movement hacks A scripting cheat that allows for highly optimised movement, resulting in movement that beyond is human ability. An example of this is bunny hopping. 4, 11, 18

Overwatch A match reviewing system within the game Counter-Strike: Global Offensive (CS:GO) that allowed trusted players to review matches with suspected cheaters. The investigators would then come to a verdict and report whether or not the suspect was cheating or not. 4, 7

Radar hack A type of cheat that allows a player to see information about the location of enemy players on the in-game minimap. 2, 4, 11

Recoil Control System (RCS) A type of scripting cheat that automatically negates the recoil effect of firing a weapon, giving perfect control over the weapon. 1, 4, 11

Server tick All information gathered throughout a game is communicated to the server in intervals of server ticks. It effectively describes how many times per second the server updates the game data. CS2 servers run on 64-ticks per second. 1, 4–6, 11

Spinbot Type of cheat where trigger bot is combined with anti-aim. This cheat makes sure that the player using a spinbot is even more difficult to be shot at. 2, 4, 11

T terrorists. 3

Trigger aim An automation tool that allows for precise aiming. However, as the mouse trigger remains manual, the reaction time of the player remains within human capability. Depending on the settings of the cheat software, trigger aim results in a high shot to kill accuracy and a natural reaction time, but may result in an unnatural aiming behaviour as the crosshair in some instances may appear to snap onto an opposing player. 1, 2, 4, 11, 18

Trigger bot An automation tool that assists precise aiming by automatically triggering a mouse click when an enemy is detected in the player's crosshair, resulting in a precise shot and natural aim. 1, 2, 4, 11, 18

Trust factor A hidden variable that aims to enhance players matchmaking experience by matching up players with similar trust factors. 2, 5, 17

VAC Valve Anti-Cheat. 1, 5, 7, 8, 10, 12–20, 27

Visual information cheats A class of cheats involving a player having visual access to information that they would not normally have. Such cheats include wall hack and radar hack. 4, 11

Wall hack A type of cheat that reveals hidden characters and objects through walls, giving an unfair advantage. 2, 4, 11

2 Introduction

2.1 First person shooters and cheating in video games

A first-person shooter (FPS) is a video game genre centered around weapon based combat experienced from a first person perspective. These games can be played both offline and online, with the latter option being particularly popular due to it's ability to facilitate competitive and cooperative multiplayer experiences.

Due to the competitive nature of FPS-games, cheating is not an uncommon occurrence. However, the concept of cheating is often subjective, as players may hold varying perspectives on what constitutes cheating. Some players might view specific actions as unfair or illegitimate, others may consider them acceptable gameplay strategies. Despite the subjective aspects of cheating, the paper *Cheating and anti-cheat system action impacts on user experience* by Bryan van de Ven identifies the three most common definitions of cheating in video games as the following:

"

1. *Using additional code or software to modify the game*
2. *Abusing faulty code in the game without using additional code or software.*
3. *Performing certain actions without using or altering code at all, such as looking at your friend's screen.*

"[2, ch. 2, p. 5]

For the purposes of this paper, cheating is defined as "*the use of additional code or software to modify the game*", as this is the most prevalent type of cheating occurring within the Counter-Strike community.

2.2 Counter-Strike

Counter-Strike (CS) is an online FPS video game franchise, that started as a modification (Mod) of the video game Half-Life[3][4]. In CS players join one of two teams: terrorists (T) and counter-terrorists (CT). The terrorists' objective is to plant a bomb at one of two designated bomb sites labelled "A" and "B", while the counter-terrorists are tasked with defending these sites and defusing the bomb if planted. Victory conditions vary by team: The terrorists win if the bomb detonates or if all counter-terrorists players are eliminated, while the counter-terrorists succeed by defusing the bomb, eliminating all terrorists players, or letting the round timer run to zero. Throughout the game, performance in terms of kills, round-wins, and -losses yields in-game currency. This currency can be used to purchase armour, weapons, and utility at the beginning of each round. Hence, the game requires precise aim, quick reflexes, economic strategy, and knowledge of the maps. This culminates in a very high skill ceiling, which endorses a competitive e-sports environment.

In September 2023 a new version of the game Counter-Strike was published, namely CS2[5]. Ever since the release of CS2, the game has had a problem with its users deploying cheats to gain unfair advantages. The use of additional software to modify the game is something the developers at Valve are aware of, since the report button features several reasons for banning a player, which include[A]:

1. Wall hacking
2. Aim hacking
3. Other hacking

Machine learning could potentially be used to detect such cheating behaviour. However, this requires a large amount of data, and no dataset with cheater annotations exists for CS2.

Creating a dataset for cheaters in CS2 is a challenging task, due to the complex nature of the data and lack of publicly available datasets on confirmed cheaters. Furthermore, there are multiple different types of cheating in online FPS games such as CS2. Additionally, distinguishing between professional level players and experienced cheaters can be very difficult, as the goal of cheating often is to appear as a legitimately skilful player.

The goal of this paper is to create a labelled dataset on CS2 cheaters, as currently there is no publicly available labelled dataset. This data is essential for training machine learning models to detect cheating behaviours in real time. Additionally, the dynamic evolution of cheats makes it problematic to have a static, non-evolving dataset on cheaters, as cheats evolve over time as well, which in turn affects player behaviour. Hence, the aim is to develop a data collection method that efficiently retrieves new data as well.

A cheat detection machine learning model was created for the previous version of Counter-Strike, namely CS:GO. Unfortunately, the dataset used for training the model is not publicly available, and the methods of obtaining labelled data (Overwatch) are no longer available with the release of CS2[6][7][8]. Pro-player data is available through the site HLTV[9]; however, this dataset only contains the data of official professional level matches; hence, no cheaters are present in this data.

This project aims to construct a labelled dataset of CS2 game data and create a method for easily retrieving new CS2 game data.

2.3 Cheats

Cheat providers generally provide software to customers through a subscription service. There are many different such services and the offered cheats include but are not limited to:

Aim hack A class of cheats involving the use of one or multiple aim and/or trigger assisting cheats. These can include but are not limited to recoil control system, trigger aim and trigger bot

Trigger bot An automation tool that assists precise aiming by automatically triggering a mouse click when an enemy is detected in the player's crosshair, resulting in a precise shot and natural aim.

Trigger aim An automation tool that allows for precise aiming. However, as the mouse trigger remains manual, the reaction time of the player remains within human capability. Depending on the settings of the cheat software, trigger aim results in a high shot to kill accuracy and a natural reaction time, but may result in an unnatural aiming behaviour as the crosshair in some instances may appear to snap onto an opposing player.

Recoil control system (RCS) A type of scripting cheat that automatically negates the recoil effect of firing a weapon, giving perfect control over the weapon.

Visual information cheats A class of cheats involving a player having visual access to information that they would not normally have. Such cheats include wall hack and radar hack.

Wall hack A type of cheat that reveals hidden characters and objects through walls, giving an unfair advantage.

Radar hack A type of cheat that allows a player to see information about the location of enemy players on the in-game minimap.

Movement hacks A scripting cheat that allows for highly optimised movement, resulting in movement that beyond is human ability. An example of this is bunny hopping: A movement style where a player continuously jumps while moving and upon landing instantly jumps again within the same server tick. CS2 uses a friction variable to slow the player down after landing, but if another jump is made upon the same server tick as landing, the player effectively removes all friction. This allows for increased movement speeds.

Spinbot Type of cheat where trigger bot is combined with anti-aim. This cheat makes sure that the player using a spinbot is even more difficult to be shot at.

Some of these cheats are extremely difficult to distinguish between just from visual gameplay alone. Hence for the data labelling, only a "cheater" and a "not cheater" label will be used. Additionally, since Valve bans players for cheating without any additional information regarding the used cheat, these labels would have to be assigned manually. However, this was believed to be beyond the scope of this project, as models trained on this type of data perform well without specific cheat labelling[8].

2.4 Demo files

The data collected in this project is all gathered in demo files(.dem). These are pure binary files that require decoding for the data to be extracted. These files contain information regarding all events and actions that occured in a match, recorded in the form of server ticks. All information gathered throughout a game is communicated to the server in intervals of server ticks. It effectively describes how many times per second the server updates the game data. CS2 servers run on 64-ticks per second.

2.5 Hypothesis

Due to the Valve Anti-Cheat (VAC)-bans being permanent and non-negotiable it is suspected that the precision of the ban-label is high, as a VAC-ban not only constitutes a ban from the particular game cheated in, but also freezes any in game items[10]. This means that a user who is banned from a game is unable to transfer their items onto another account. These items include but are not limited to:

1. Weapon skins
2. Agent skins
3. Music kits
4. Glove skins
5. Charms
6. Stickers
7. Sprays
8. Cases

Note that some items within these categories can sell for thousands of euros on the Steam Community[11]. Hence, the consequences for receiving a ban can potentially lead to the loss of high-priced items. Based on this observation, it can be hypothesised that achieving high precision in identifying and banning cheaters aligns with Steam's best interests. However, this focus would likely lead to a reduced quality in the classification of users labelled as "not cheater". Another contributing factor to label accuracy is trust factor[12]: A hidden variable that aims to enhance players matchmaking experience by matching up players with similar trust factors. It is hypothesised that cheaters will likely have low trust factor, meaning that they are more likely to be matched up against or with other cheaters. Steam has chosen not to publish the underlying algorithm that is used to calculate the trust factor, hence this is not possible to verify that players have a low trust factor.

3 Related work

This project is conducted with the objective of contributing to a thesis that explores the utilization of the generated dataset for training a machine learning model designed to detect cheating behaviour. In terms of cheat detection in FPS-games most papers utilise computer vision methods on the first person view in order to determine whether a player is conducting cheater like behaviour[13][14]. However, due to Counter-Strike being made using the Source 2 engine, events in the form of server ticks can be recorded in a demo file[15]. In the context of CS these demo files represent a match which at a later time can be rewatched. Due to the accessibility of this type of data, papers covering cheat detection regarding CS typically use demo files for this purpose[8].

An addition to the project may involve manipulation of the data. Without explicit approval from users participating in this data gathering, the dataset will not be published. However, anonymising the data may allow for the publication of the dataset. Valve provides the means to parse a demo file through the *csgo-demoinfo* tool[16]. However, this tool has not been updated since 2017 and the most recent issue discusses new errors introduced by the launch of CS2[17]. A plentiful array of alternative solutions created by third parties exist, however, none of these alternatives offer the possibility of altering the demo file[18][19][20][21]. Hence, a future project could entail modifying one of these demo file parsers in such a way that anonymity could be ensured for the players. This would entail censoring usernames and any mentions thereof, removing chat history, removing players skins and any other data that could be used to link to a specific player or their profile.

The task is challenging as demo files are pure binary files and Valve does not provide documentation on how these files are created. Decoding these files would involve reverse engineering a demo file decoder by determining what information is stored and in what order, then censoring any personal data, and finally encoding the file again.

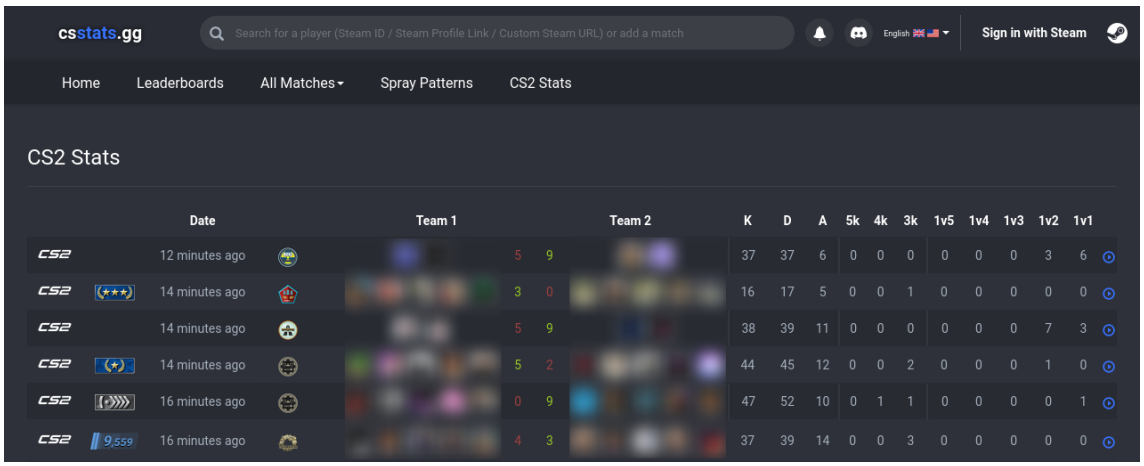


Figure 1: A screenshot of the "all matches" page of csstats.gg, with all profile pictures blurred[29](Taken the 7. dec 2024)

4 Methodology

4.1 Gathering data

In order to gather match data, one needs access to the data itself. This is challenging as matches, that are played on the Valve servers, aren't shared through any official sources. Other works utilise access to match data via the Overwatch service provided by Steam[8]. In the context of Counter-Strike, Overwatch refers to a match reviewing system, available in CS:GO. This feature allowed trusted players to review matches with suspected cheaters. The investigators would then come to a verdict and report whether or not the suspect was cheating or not. Although there was no official route to verify whether a verdict had resulted in a VAC-ban, a project reverse engineering player IDs was devised[22]. Unfortunately, the Overwatch feature was removed 22/3/2023 as the CS2 limited tests began. Version 1.38.5.5 remains the last version of CS:GO, where this feature was available, and has as of December 2024 not been reintroduced[6][23][24][25][7].

4.1.1 Source of data

The lack of an official data source was mitigated by web scraping a match sharing service. A match sharing service is a service provided by a third party, where users can upload their matches. The service often provides the means of analysing play styles enabling users to improve performance. The matches are shared via match sharing codes, which is a string of the format:

```
CSGO(-[a-zA-Z0-9]{5}){5}$
```

As a guide for the reader, the regular expression above results in a match sharing code like the following:

```
CSGO-xxxxx-xxxxx-xxxxx-xxxxx-xxxxx
```

, where the "x"s are alphanumeric characters. These codes can be retrieved from CS through the game user interface. Note that when the code is retrieved it is contained in a Steam link, which automatically opens the game and downloads the match. The match sharing codes can then be uploaded to a match sharing service either by retrieving it manually or by allowing a service to retrieve it for you by logging onto the site via. Steam login. The site then uses the Steam user's API key to fetch the matches automatically[26]. Using the CS2 user interface or a dedicated server method would require downloading matches manually one by one. This would have become too time consuming so alternatives were explored. One such alternative was CS Demo Manager[27], which is an open-source application that allows for CS2 demo file management and statistical analysis of gameplay within a demo file. However, it also provides a command line interface for downloading demo files in bulk using match sharing codes[28]. This tool was used to download all data for this project.

The platform csstats.gg is an example of a match sharing service. csstats.gg has an "All matches"-page, as seen in Figure 1, which showcases the latest 50 matches that have been uploaded to the

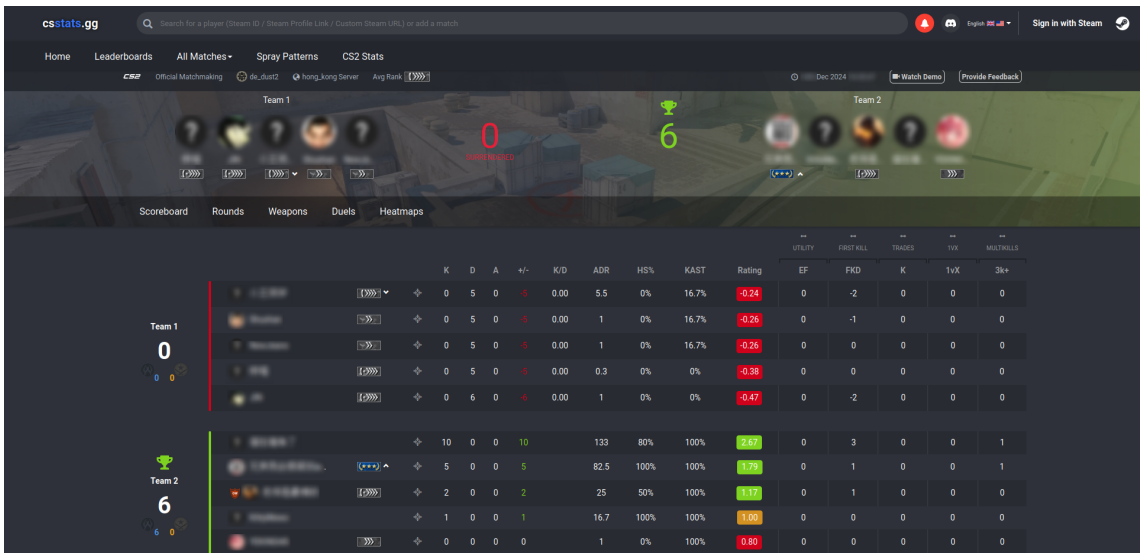


Figure 2: csstats.gg match page view

Module name	Description	
BeautifulSoup	A library for HTML parsing functionality	[35]
cloudscraper	A library for bypassing Cloudflare's anti-bot measures	[34]
collections	A module for container data types	[36]
csv	A module for csv file read-/write functionality	[37]
datetime	A module for manipulating dates and times	[38]
math	A module for mathematical operations	[39]
os	A module for operating system functionality like directory navigation	[40]
pandas	A library for data manipulation and analysis	[41]
pygame	A library for python game development	[42]
time	A module for measuring and converting time formats	[43]
re	A module for regex functionality	[44]

Table 1: The names of the packages and modules used in this project and a short description

site[29]. Each entry in the "All matches"-table provides a link to their respective match pages. The match pages have a "watch demo" button in the top right corner as seen in Figure 2, which if pressed, opens a Steam link which starts a download of a demo file. No matter the time of upload to the site, if the match has been played within the past 30 days, any user is able to download the demo file of the match. If the match is older than 30 days, the match sharing code has expired and the match can not be retrieved from the CS match sharing servers. Since csstats.gg only saves specific data points additionally to the match sharing code, accessing the demo files within 30 days is essential. The access to demo files through this site was used as the source of data for this project.

One potential alternative data source for "not cheater" games could have been the third party match provider website Faceit[30]. Their matches have their own elo system and the servers require the use of a more intrusive anti-cheat software. Furthermore, Faceit has it's own anti cheat system, meaning cheaters get penalised by a Faceit-ban rather than a VAC-ban[30]. However, this data source was not utilised due to the format of Faceit matches being slightly different from Valve matches adding new complexity to the data.

4.1.2 Data Scraping

In the interest of time, retrieving match sharing codes from the site required automation. Hence, a data scraper was created. The source code of the data scraper can be found in the Github repository CS2-demo-scraper[31]. The data scraper was written in Python 3.11[32] using the libraries as seen in Table 1. Since csstats uses the website protection software Cloudflare[33], retrieving information from the site required precaution. For this reason the python library cloudscraper[34] was used.

The cloudscraper library is designed to bypass Cloudflare's fundamental anti-bot protection mech-

anisms. While this functionality is adequate for retrieving standard HTML webpages, it is insufficient for accessing Steam links containing match sharing code. When requesting a demo file, the site employs a more advanced security measure, requiring the completion of a Turnstile Completely Automated Public Turing test to tell Computers and Humans Apart[1] (CAPTCHA) to proceed. Successfully solving the CAPTCHA updates the `cf_clearance` cookie, which grants the session access to the data for 30 minutes. When the 30 min are over a new Turnstile CAPTCHA needs solving to be granted continued access. Although tools capable of automating CAPTCHA solving are available, most require payment and are not open-source[45][46][47][48][49]. Consequently, we chose to manually resolve the CAPTCHA at 30-minute intervals and update the `cf_clearance` cookie to ensure continued access. As an aid to the person conducting the data gathering, a tune plays when the cookie has expired, hence the use of the python library pygame[42]. Consequently, the data scraping was mostly done during European daylight hours, which potentially could lead to the gathered data mostly originating from the Eurasian continent. Furthermore, in order to not overload csstats with requests, the data scraper used a sleep timer, which was uniformly distributed in the range of 5 to 10 seconds. This sleep timer ensures that the data scraper isn't registered as a threat by Cloudflare, which could potentially result in an IP-ban of unknown duration.

4.1.3 Scraping Algorithm

The player data stored on csstats is closely interconnected, as a match is composed of multiple players and players can participate in multiple matches. Hence, the connectivity between players can get quite dense. In the interest of time, it is not possible to fully explore the deeply connected graph that is the player relations. Additionally, due to the structure of csstats, the "All matches"-page is the only entry point into the data not requiring any prior knowledge. In order to gain access to more than 50 games every 30 minutes, an algorithm was devised. The algorithm, as shown in Algorithm 1, iterates through all matches on the "All Matches"-page. Each participant's match history is then iterated through and appended to a file, given that it does not appear in the file already, is less than 30 days old and is of the correct match type.

Algorithm 1 Web scraping

```
file ← getCSVFile()
games ← getMatches("all match page")
for each g in games do
    players ← g.getPlayers()
    for each p in players do
        history ← getPlayerHistory(p)
        for each match in history do
            if (match.id ∉ file) ∧ (match.age ≤ 30) ∧ (match.type ≠ "wingman") ∧
            (match.type ≠ "faceit") then
                file.append(match)
            end if
        end for
    end for
end for
```

4.1.4 Web Scraped Data

Due to csstats storing useful data for every match, data additional to the match sharing codes was scraped, as it could prove useful as an indicator of data variance. Therefore, the following data from each match was saved in a csv file:

1. csstats match_id
2. steamlink
3. map
4. avg_rank
5. type

Match type	Scraped
CS:GO Official Matchmaking	No
Faceit Matchmaking	No
CS2 Official Matchmaking	Yes
CS2 Premier Matchmaking	yes

Table 2: The match types hosted by csstats and whether they were scraped or not

6. team1_string
7. team2_string
8. cheater_names_str
9. demo_file_name

The csstats match ID was saved due to the ease of revisiting matches and verifying that players participating in the match remained unbanned. The Steam link was saved as this contained the match sharing code needed to download the demo file. The map and average rank of players was saved in order to potentially preserve variance in the dataset at a later point. The type, referring to type of match making, was stored, as different types of match making may impact cheater frequency. For a comprehensive list of match types saved see Table 2. The type was saved due to the fact that only matches with 10 participants are within the scope of this project. Hence, the need to filter wingman matches as these matches utilise a different map pool and only have 4 participants. Furthermore, the site displays Faceit games as well, however due to reasons stated in section 4.1.1, these matches were not included. The team1_string and team2_string contain the csstats player ids of the participants. This information was saved in order to easily verify that a match had 10 participants, as it is possible to encounter players with deleted Steam profiles, in which case they are just named "player (n)", where n is a number in the range 1 to 5. These games were not included as the reason for the deletion of these profiles were unknown. However, it is speculated that it may have had to do with illegitimate activity. The usernames of the VAC-banned players was saved as well in order to easily isolate "cheater" data. The demo file name was added to the csv file after a certain match had been downloaded.

Furthermore, with the nature of cheating as defined in subsection 2.1 it is possible that a VAC-banned user has played games where they had not displayed any cheating behaviour. Combined with the irreversibility of a VAC-ban, means that the dataset could contain cases, where a player, who cheated, and subsequently got banned, has previous games where they had not cheated. In this case, the game where no cheating took place would be labelled as a cheater game. The extent of this problem will be discussed in section 4.2.

4.2 Manual review and data quality

In order to determine the quality of the data labels, a manual data review was conducted on two subsamples of the dataset. A subset for the matches with at least one banned user present, and one for the matches where no users have been banned within 2 days of the web scraping and download taking place. This makes the potential age of the matches to be in the range of 2-32 days old. The subsets are identical in size with 50 demo files being reviewed in each category, making the total number of demo files reviewed 100.

4.2.1 Manually detecting cheaters

The question arises: how can one identify a cheater? While it is impossible to determine with absolute certainty whether a user is engaging in cheating behaviour, the following is a compilation of techniques that can be employed to potentially detect instances of cheating. Although these methods cannot definitively classify a player as a cheater, they may serve as strong indicators suggesting the likelihood of cheating behaviour. These methods fall into the following categories: Observation based analysis and statistical analysis.

4.2.1.1 Observation based analysis One approach of manually detecting cheating is through observing player behaviour during gameplay. This is accomplished by reviewing the recorded

gameplay stored in a demo file through the CS-watch feature. Indicators of cheating behaviour detected using this method includes the following:

Unnatural aim This behaviour involves atypical movements of the crosshair/aim, but does not necessarily result in high kill accuracy. This behaviour includes aiming at angles that would under normal circumstances not provide a strategic advantage, such as aiming at walls or only guarding one angle, despite the opposing team being able to approach from multiple directions. Such behaviour often aligns with the use of cheats like: visual information cheats and wall hack.

Unnatural Precision This involves a level of precision that exceeds normal human capabilities or appears mechanically enhanced. Specific examples could include an unusually high number of headshots. Such behaviour often aligns with the use of cheats like: recoil control system, trigger aim, and trigger bot

Unnatural movement The player displays uncharacteristic or exaggerated movements. For example, a spinbot cheat causes the player's in-game character to spin rapidly while maintaining full gameplay functionality, making the movement highly unnatural. Another example could be the player perfectly executing bunny hopping, as the window of opportunity to execute the next jump is only 0.015625 seconds (1 server tick). However, players with movement hacks do bunny hopping without failure for minutes at a time.

Predictive Playstyle The player demonstrates an uncanny ability to predict opponent positions or actions, suggesting the use of cheat software such as: wall hack, radar hack, and visual information cheats.

Skill gaps The player appears as highly skilled in one area of the game, but displays a significant difference in skill level in another area. For example a player can have a high headshot precision indicating that they are an experienced player however, the player may be very unskilled in using utility. This discrepancy in skill level can suggest the use of skill enhancing cheats such as: trigger aim, trigger bot, and any type of movement hacks

4.2.1.2 Statistical analysis

A second method for manually detecting cheating behaviour is through statistical analysis. This is done by analysing information extracted from a match. Fortunately, csstats has a match overview, where this type of information is displayed. Note, that this type of analysis often relies on the player performing well as a result of deploying cheats, however, simply performing well does not constitute cheating. Similarly, performing poorly does not exclude the possibility of a player using cheats. Indicators of cheating behaviour detected using this method includes the following:

High kill to death ratio A high kill to death ratio is the result of killing a high number of targets in comparison to the number of times the player has died. This indicates that a player is performing well, however further investigation is needed before a cheater-label can be assigned.

Headshot to kill ratio A headshot is a shot directly to the head of the target dealing approximately 400% of the weapons base damage often resulting in an instant kill[50]. Due to the high damage dealt, these types of shots are advantageous, however due to the small hitbox of the target, it is an acquired skill to perform these shots when taking weapon recoil into account. Therefore, a high number of headshots in comparison to other types of kills may indicate the use of cheats such as: trigger aim, trigger bot and recoil control system.

High ratio of wall bang kills A wall bang is a shot through a wall that hits a target. Wall bangs can have strategic purposes, however killing a target via wall banging is quite uncommon. Hence, a high number of wall bangs may allude to the fact that a player is conducting cheating behaviour such as: trigger bot and wall hack.

High ratio of kills through obscurant smoke In CS2, the smoke grenade, commonly referred to as a "smoke", is a utility item that generates an opaque cloud upon deployment. This smoke is designed to obstruct vision and can only be partially disrupted by explosive actions, such as shooting or deploying explosive grenades directly at it. Consequently, a disproportionately high number of kills achieved through the smoke may indicate potential cheating behaviours, such as the use of wall hack, radar hack, and visual information cheats. It can be argued that a team with good communication skills may be able to generate a large amount

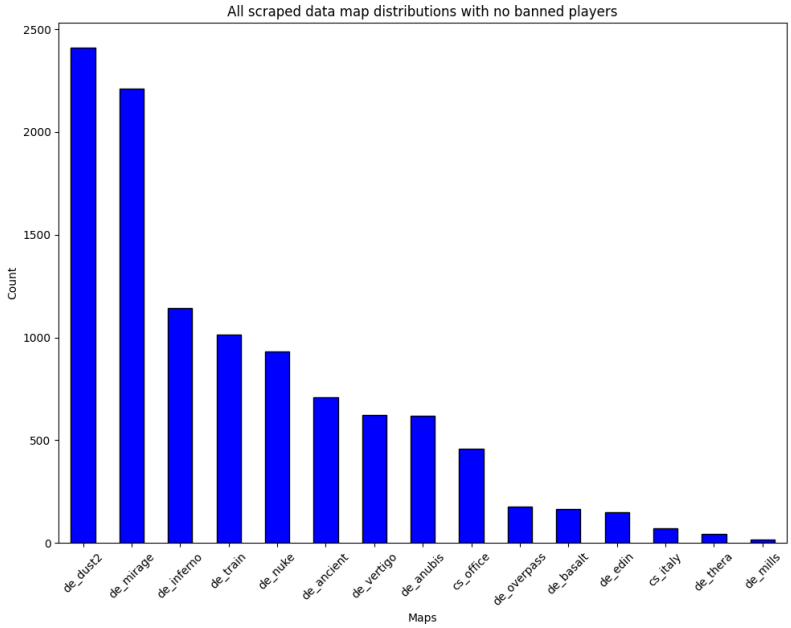
of kills through obscurant smoke. When an explosive grenade detonates within a smoke, a temporary window through the smoke is created. Killing an enemy by peeking through such a window counts as a "smoke grenade kill" I.E. a kill through the smoke.

4.2.2 Reviewing gameplay

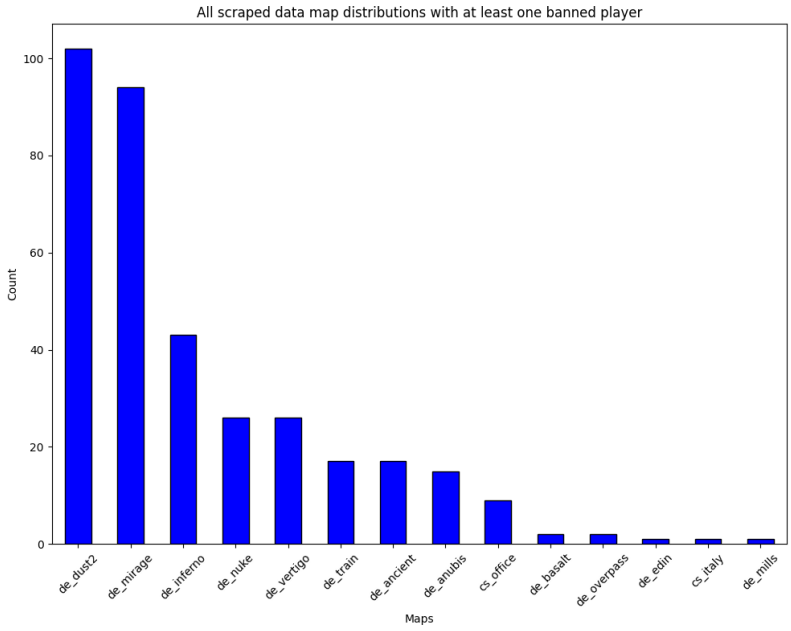
With the guidelines for manually detecting cheating behaviour as outlined in section 4.2.1 it is possible to conduct a review of the cheater label as assigned by VAC-bans. From a csv file with data gathered as specified in section 4.1, 100 games were chosen at random with 50 of the games containing at least one VAC-banned player, and the other 50 not containing VAC-banned players. The size of the sets was chosen due to the laborious process of manually labelling data. Furthermore, it is noteworthy that it is less time consuming to manually label cheating behaviour as opposed to non-cheating behaviour due to cheaters exhibiting multiple signs of cheating behaviour, however users not deploying cheats may also exhibit some of these signs. With the "not cheater" label making up the majority of the data, a set of size 100 was deemed appropriate with the projects given time frame. For each player in every game in each set a manual review was conducted. The label and the actual label was noted in separate csv files: one for the set with no banned players present and one for the set with at least one banned player present. After the review was completed a confusion matrix containing the results was constructed for each set which can be found in section 5.1.

5 Data format and structure

Performing 30 hours of data scraping of the website csstats.gg as described in section 4.1 resulted in 11 088 match sharing codes. Of the matches represented by the match sharing codes, 10 732 of the matches had no VAC-banned players and 356 had at least one VAC-banned player creating approximately a 30 to 1 ratio of games with no banned players versus games with banned players. Using the selection of data scraped as described in section 4.1.4, it appears that there is a natural bias towards certain maps such as de_dust2 and de_mirage as seen in Figure 3. Some maps being played more than others could be the result of popularity, however some maps such as cs_italy and de_mills have been limited releases, while maps such as de_dust2 and de_mirage may be more popular due to them as of December 2024 being in the active duty map pool[51].



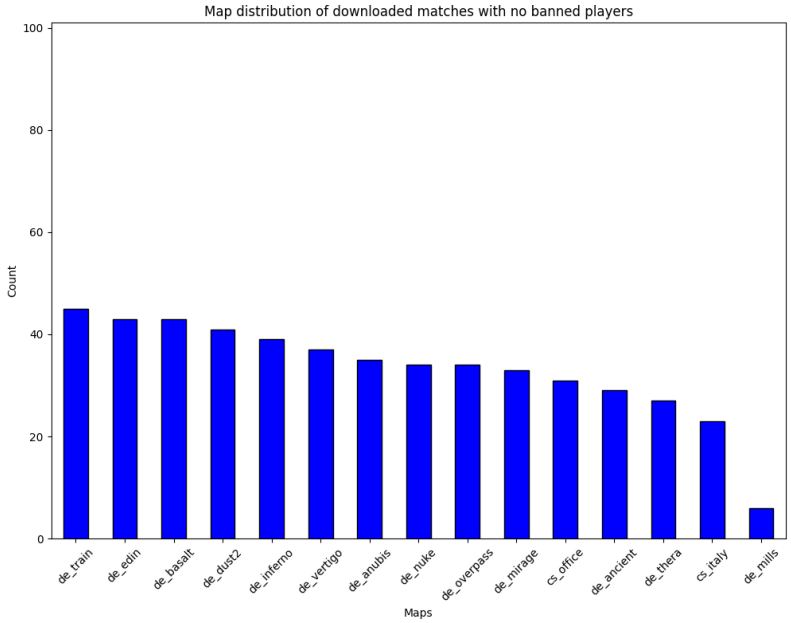
(a) [n=10 732] No VAC-banned players present



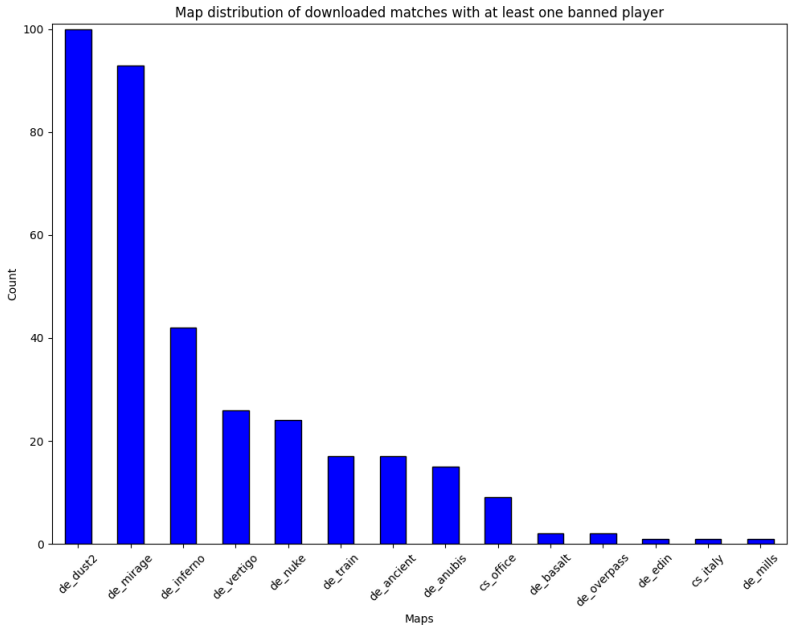
(b) [n = 356] VAC-banned players present

Figure 3: Map distribution for all data scraped

Unfortunately, due to a limitation in storage capacity, only a subset of these match sharing codes were downloaded. In the interest of keeping the size of the cheater and non-cheater set somewhat balanced, all of the matches with at least one VAC-banned player (350) were downloaded while select matches with no banned players were kept. The attentive reader may notice that Figure 3b mentions 356 match sharing codes being scraped, however only 350 demo files were downloaded. This is due to 6 match sharing codes expiring in the short period between scraping and download. Since maps vary play style and thereby player behaviour an attempt at balancing the downloaded maps with no banned players present was made, which can be seen in Figure 4a. Furthermore, in order to not introduce additional noise, matches with less than 10 players were not selected in this process. This resulted in 500 matches without banned players being selected for download, making the total number of demo files downloaded 850. Hence, the total storage space needed to store these files was approximately 178 GB.



(a) [n = 500] No VAC-banned players present



(b) [n = 350] VAC-banned players present

Figure 4: Map distribution for downloaded data

In terms of data origin, by far the most data originated from Europe, as seen in Figure 5. Counting all EU based servers, data collected from Europe made up 61.4% of all data[B]. This may partially be due to the fact that most Steam servers hosting CS2 matches are located in Europe and Asia. However, due to data scraped from Asia making up 17.9% it is evident that this is not the only reason. Another possible explanation could be due to the data scraping mainly taking place during European daylight hours. A third reason could be a lack of linguistic accessibility to csstats.gg as the site mainly targets English speakers. All tough csstats.gg does provide some aid for non-English speakers, this service only covers the front page. Since at least one user in a match needs to sign up to this service in order for a match to be tracked on the site, this hypothesis may hold as more data was gathered from the Americas (19.7%) than Asia (17.9%), despite Asia having more local servers [B][52]. Note, that this does not take the computing power of the individual server locations into account.

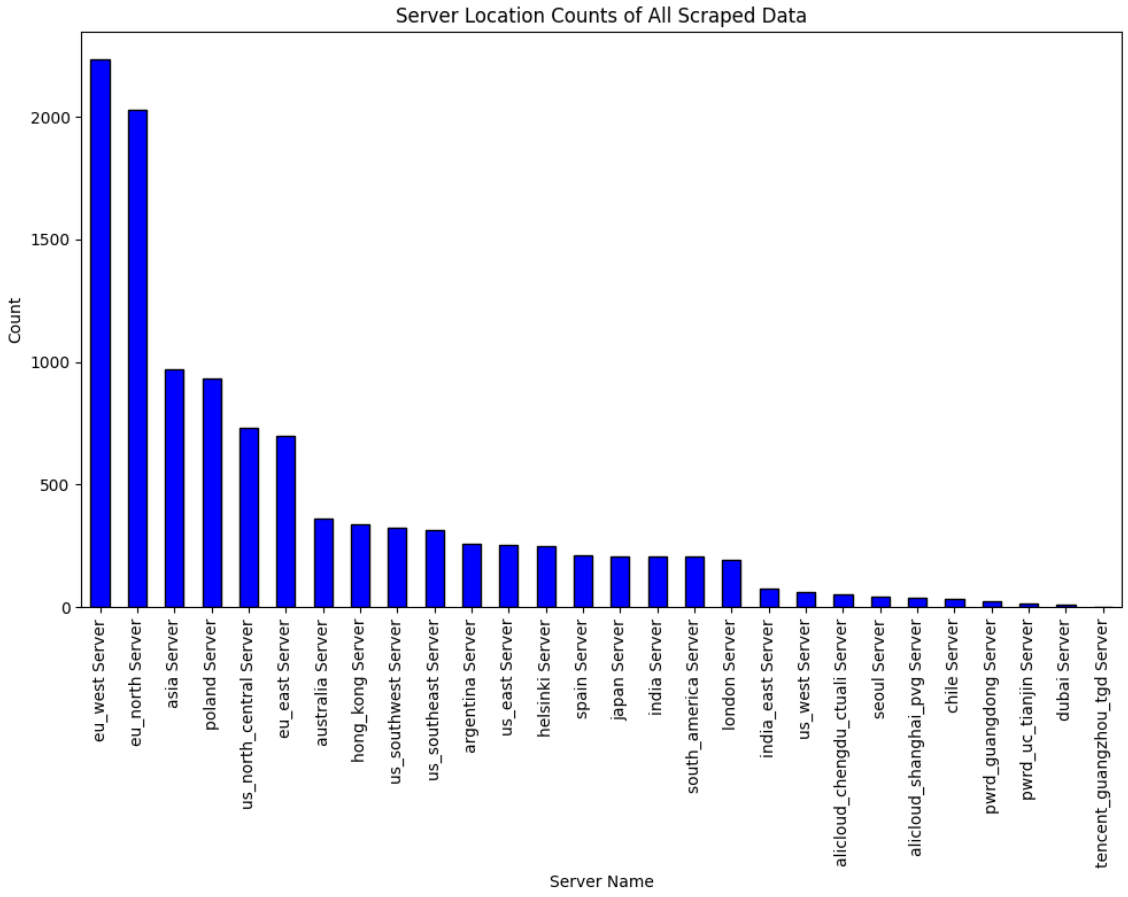
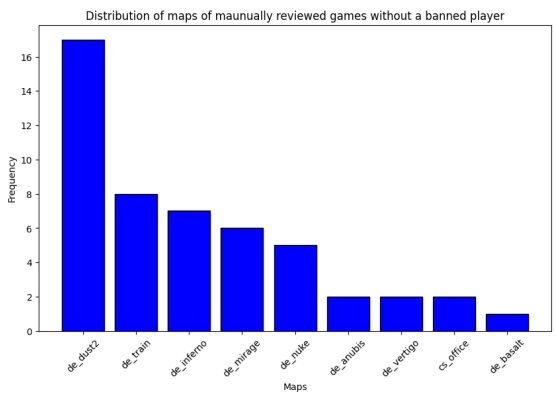


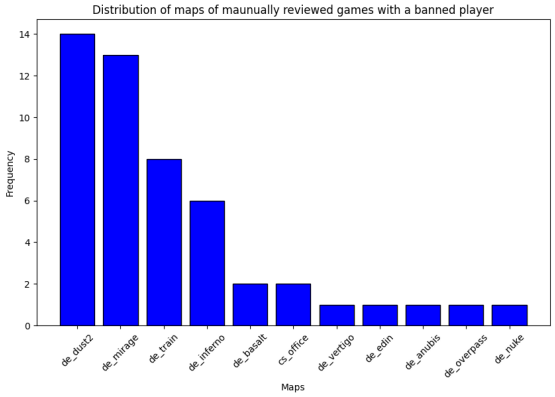
Figure 5: Server locations of all scraped data

5.1 Results of manual data review

Performing a review of the data labels as described in section 4.2 resulted in two random subsets of demo files being chosen for a review: one with at least one banned player and one without. The map and rank distribution of matches with no VAC-banned players present can be seen in Figure 6a, 7a, and 8a. Likewise the map and rank distribution for matches with VAC-banned players present can be seen in Figure 6b, 7b, and 8b. Note that the random samples were chosen from the total data scraped and not the reduced set of downloaded matches. Hence, the map distribution of matches with no VAC-banned users may highly resemble that of the total distribution rather than the uniform map distribution in the downloads. One noteworthy point about these results is that VAC-banned players had higher ranks in both premier and non-premier matchmaking. This could suggest that cheaters are able to succesfully gain higher average ranks due to unfair advantages.

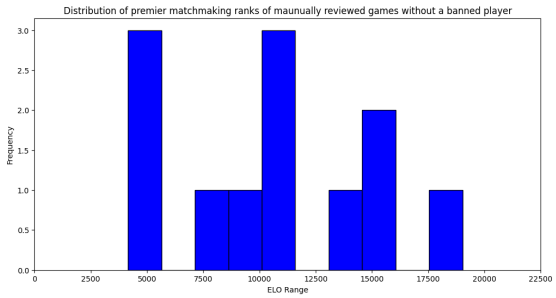


(a) [$n = 50$ no VAC-banned user(s) present]

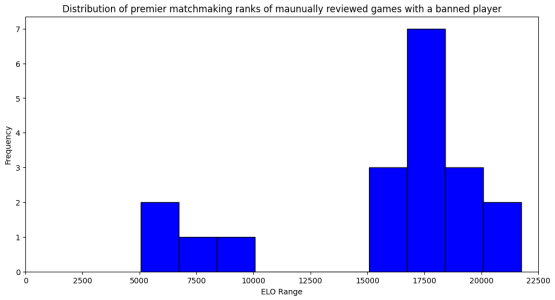


(b) [$n = 50$ VAC-banned user(s) present]

Figure 6: Distribution of maps in manual review

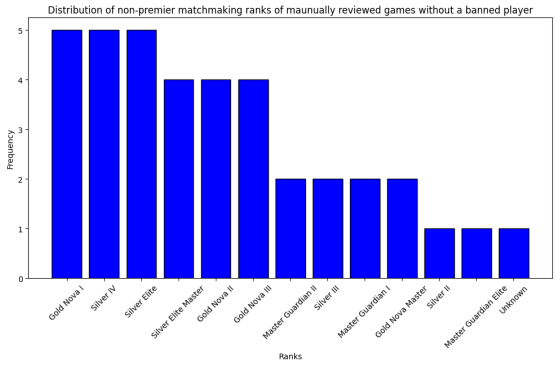


(a) [$n = 50$ no VAC-banned user(s) present]

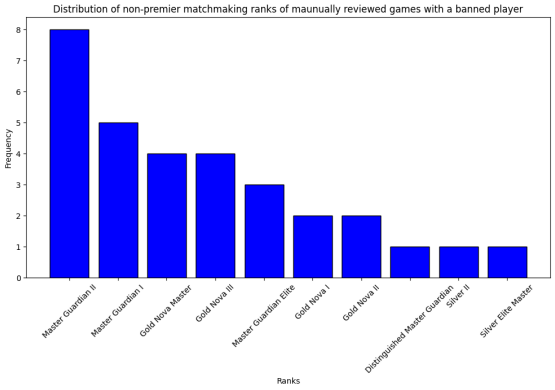


(b) [$n = 50$ VAC-banned user(s) present]

Figure 7: Distribution of premier ranks in manual review



(a) [$n = 50$ no VAC-banned user(s) present]

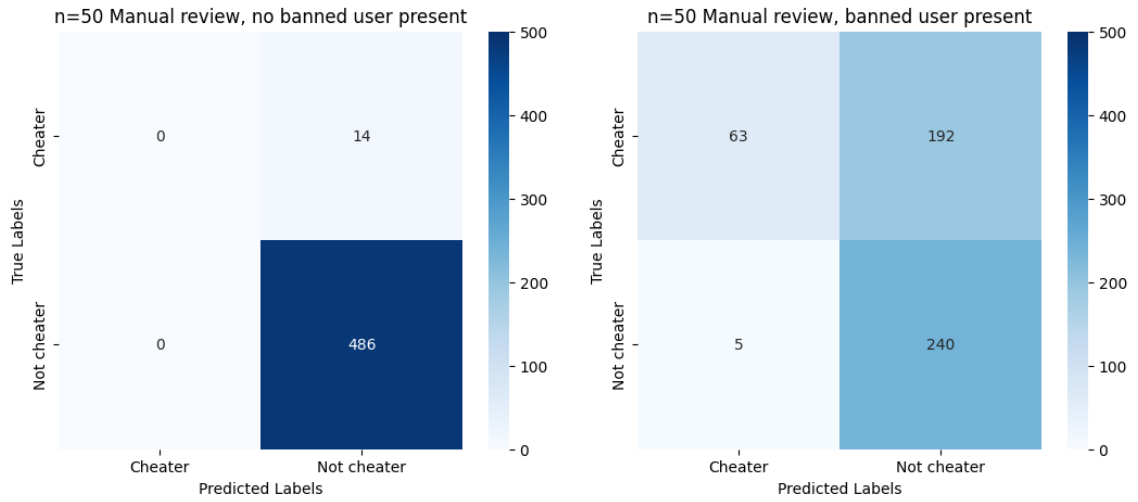


(b) [$n = 50$ VAC-banned user(s) present]

Figure 8: Distribution of non-premier ranks in manual review

5.1.1 Quality of data labels

In order to determine the quality of the label two confusion matrices are created as seen in Figure 9. The figure is divided into two subgraphs, one describing the results of the subset with at least one banned user present, and one with no banned users present. In the set of matches with no banned users as seen in Figure 9a, there are no predicted cheaters I.E no VAC-banned users registered for these matches within a 2-32 day period. However, when reviewing these matches, 14 out of 500 players were expressing cheating behaviour beyond a reasonable doubt, making the precision of the "not cheater" label in matches with no cheaters 97.2% as seen in Table 3a. Arguably a precision of 97.2% is not bad, however, this implies that further outlier detection should be performed before this dataset is to be used. Furthermore, when inspecting Figure 9b it is apparent that the "not cheater" label is of worse quality, with 192 out of 432 players expressing cheating behaviour beyond a reasonable doubt. This resulted in a "not cheater" label precision of 55.6%



(a) [$n = 50$ no VAC-banned user(s) present], The confusion matrix resulting from a manual review of 50 demo files with where no VAC-banned user was present

(b) [$n = 50$ VAC-banned user(s) present], The confusion matrix resulting from a manual review of 50 demo files where at least one VAC-banned user was present

Figure 9: Confusion matrices resulting from the manual review

for the dataset with at least one banned user in the matches while the precision of "cheater" label in the same dataset is 92.6% as seen in Table 3b. The stark difference between the precision of the "not cheater" label and the "cheater" label in this dataset may be due to Steam using trust factor match making, meaning players with a low trust factor are matched with other players of a low score as well[12]. This increases the likelihood that, in a game with at least one VAC-banned player, other cheaters—who may not yet be banned—are also present, as cheaters tend to have low trust factor scores.

	Precision	Recall
Not cheater	0.972	1
Cheater	0	0

(a) The dataset with no banned players

.

	Precision	Recall
Not cheater	0.556	0.98
Cheater	0.926	0.247

(b) The dataset with at least one banned player

.

Table 3: The precision and recall values calculated for the datasets used in the manual review[C]

.

5.1.2 Cheater behaviour

Review of the matches with VAC-banned player(s) revealed multiple behavioural tendencies of cheaters, which differ significantly from non-cheaters.

Firstly, cheaters gravitate towards weapons that deal a lot of damage with a single bullet, such as the R8 revolver, SSG 08 and the AWP. This is due to them using some form of aim hacking, such as trigger aim or trigger bot. Using these aim hacks combined with weapons that can kill an enemy in a single bullet is a very effective strategy, as they become nearly unstoppable. Secondly, cheaters move in unnatural ways by using movement hacks. Most commonly a bunny hopping script is used. This type of movement is unachievable for non-cheating players, as the timing of a bunny hop is extremely difficult to do. However, players with movement hacks do bunny hopping without failure for minutes at a time and are therefore easy to distinguish.

6 Reflection

This project aimed at creating a dataset for CS2 with cheater annotation. With a total of 11 088 matches collected with a label precision of 97.2% and 92.6% for non-cheaters and cheaters respectively, it is fair to say that this goal has been met in some capacity. However, with a ratio of 30 to 1 in terms of games with no cheaters in contrast to games with cheaters present it is necessary to mention that optimisations of the data gathering methods were possible. Since the data heavily skewed towards non-cheater data, only a fraction of the games with no VAC banned players were downloaded. This could have been mitigated by making a second scraping algorithm that would target games with banned player(s). This could have saved scraping time by targeting the data that is more scarce.

In terms of the quality of data, the precision of the "not cheater" label, all though not bad, could have potentially been improved by scraping games from Faceit matches. This is due to Faceit using an invasive anti-cheat software, which makes cheating much more unlikely[53]. As stated in section 4.1.1, Faceit matches were considered for this project. However, due to Faceit matches having a different format than matches on official CS2 servers, they were excluded from the dataset.

Finally, a potential enhancement of the data scraping algorithm could involve the utilisation of FlareSolverr as this open source project allows for automated solving of Cloudflare Turnstiles[54]. This would mitigate the need for manually updating the `cf_clearance` cookie. Using this tool in the project could potentially also lead to a more balanced set in terms of data origin, as most of the data in the data set originates from the Eurasian continent.

7 Conclusion

Using a match sharing service such as csstats in order to gather CS2 game data in the form of match sharing codes proved possible through the means of a python script using several libraries with the most important one being cloudscraper[34]. Scraping for 30 hours with a sleep timer uniformly distributed in the range of 5 to 10 seconds yielded 11 088 data points. Note that the results are not from a singular session as the scraping required manual upkeep as described in section 4.1.2. Consequently, the scraping was done during European daytime hours. It is possible that this may have skewed the origin of data towards the Eurasian continent, however the causality of these events is up for debate as the majority of CS-servers are located in these regions[52]. This bias towards the Eurasian continent could possibly be mitigated by the use of the open-source Cloudflare Turnstile solver Flaresolverr[54].

From the matches that were scraped, approximately 1 in 30 had a VAC-banned player. This resulted in a heavy bias towards what is labelled as "not cheater" games. Due to this imbalance all matches containing a VAC-banned players were downloaded, but only a subset of matches with no VAC-banned players were downloaded. This resulted in 350 matches with at least one VAC-banned player present and 500 matches with no VAC-banned players present. This totals to 178 GB of downloaded and saved labelled data. Due to the abundance of non-cheater matches, the selected matches from this dataset aimed at preserving variance in terms of map distribution, as the difference in maps introduce different play styles. Though it could be argued that a difference in play style can also be introduced by skill level, this was not opted for due to the data gathering scraping both CS2 competitive and premier matches. These two types of skill measurements are inherently incomparable, and requires new analysis in order to determine alignments in ranks. This was deemed outside the scope of this project.

Due to the volume of data gathered, a manual data review was conducted on a subset of demo files in order to investigate the precision of the VAC-ban label and whether users with the VAC-ban label conducted cheater behaviour. The precision of the label turned out to be 92.6% however, recall was 24.7%, which can be further explored by the precision of the "not cheater" label being 55.6% for this particular dataset. In this case it means that non VAC-banned players in a game with at least one VAC-banned player are likely to be cheating as well, but not yet banned. Therefore, it is recommended to use an "unknown" label for all non VAC-banned players in a game with at least one VAC-banned player present. Manual review of games with no VAC-banned players present yielded a "not cheater" label precision of 97.2%, meaning in a game with no VAC-banned players it is far more unlikely to come across an unbanned cheater.

In summary, this project highlights both the opportunities and challenges of gathering labelled gameplay data from the match sharing service csstats. While the collected dataset provides a foundation for further analysis, it is best to be cautious when using the data for training of machine learning models as outliers occur within the dataset. Future work could explore automated methods to enhance label reliability and address geographic biases in data collection.

8 References

[1] Cloudflare. *How CAPTCHAs work — What does CAPTCHA mean?* URL: https://www.cloudflare.com/learning/bots/how-captchas-work/. (accessed: 11.dec 2024).

[2] Bryan van de Ven. "Cheating and anti-cheat system action impacts on user experience". Available at https://www.cs.ru.nl/bachelors-theses/2023/Bryan_van_de_Ven___1024205___Cheating_and_anti-cheat_system_action_impacts_on_user_experience.pdf. Master's thesis. Radboud University, 2023.

[3] Valve. *Half-Life*. URL: https://www.half-life.com/en/halflife. (accessed: 4.dec 2024).

[4] Valve. *Counter-Strike 2*. URL: https://www.counter-strike.net/. (accessed: 4.dec 2024).

[5] Valve. *Counter-Strike 2 Release*. URL: https://steamcommunity.com/games/CSGO/announcements/detail/3747614808335403092. (accessed: 10.dec 2024).

[6] Counter-Strike Wiki. *Overwatch*. URL: https://counterstrike.fandom.com/wiki/Overwatch. (accessed: 2.dec 2024).

[7] Valve. *Counter-Strike: Global Offensive Overwatch FAQ*. URL: https://blog.counter-strike.net/index.php/overwatch/. (accessed: 3.dec 2024).

[8] Harry Dunham. "Cheat Detection using Machine Learning within Counter-Strike: Global OffensiveGlobal Offensive". Available at https://openworks.wooster.edu/cgi/viewcontent.cgi?article=11803&context=independentstudy. Senior Independent Study Theses. The College of Wooster, 2020.

[9] HLTV. *Counter-Strike News & Coverage*. URL: https://www.hltv.org/. (accessed: 2.dec 2024).

[10] Steam. *Valve Anti-Cheat (VAC) System*. URL: https://help.steampowered.com/en/faqs/view/571A-97DA-70E9-FF74. (accessed: 4.dec 2024).

[11] Steam. *Steam community market :: Listings for StatTrack Karambit*. URL: https://steamcommunity.com/market/listings/730/%E2%98%85%20StatTrak%E2%84%A2%20Karambit.

[12] Steam. *CS2 - Trust Factor Matchmaking*. URL: https://help.steampowered.com/en/faqs/view/00EF-D679-C76A-C185. (accessed: 4.dec 2024).

[13] Bin Ma Binghua Nie. "VADNet: Visual-Based Anti-Cheating Detection Network in FPS Games". Available at https://www.iieta.org/journals/ts/paper/10.18280/ts.410137. MA thesis. School of Information Engineering, North China University of Water Resources and Electric Power, 2024.

[14] Aditya Jonnalagadda et al. "Robust Vision-Based Cheat Detection in Competitive Gaming". Available at https://dl.acm.org/doi/pdf/10.1145/3451259. MA thesis. University of California, 2021.

[15] Valve Developer Community. *DEM (file format)*. URL: https://developer.valvesoftware.com/wiki/DEM_(file_format). (accessed: 6.dec 2024).

[16] ValveSoftware. *csgo-demoinfo - Counter-Strike: Global Offensive - Demo Parsing Tool*. URL: https://github.com/ValveSoftware/csgo-demoinfo. (accessed: 6.dec 2024).

[17] Doeke Wartena. *Does not work in CS2 - Issue #40 ValveSoftware/csgo-demoinfo*. URL: https://github.com/ValveSoftware/csgo-demoinfo/issues/40. (accessed: 6.dec 2024).

[18] Peter Xenopoulos. *pnxenopoulos/awpy : Python library to parse, analyze and visualize Counter-Strike 2 data*. URL: https://github.com/pnxenopoulos/awpy. (accessed: 10.dec 2024).

[19] Saul Rennison. *saul/demofile : Node.js library for parsing Counter-Strike: Global Offensive demo files*. URL: https://github.com/saul/demofile. (accessed: 10.dec 2024).

[20] LaihoE. *demoparser - Counter-Strike 2 replay parser for Python and JavaScript*. URL: https://github.com/LaihoE/demoparser. (accessed: 6.dec 2024).

[21] StatsHelix. *StatsHelix/demoinfo : A library to analyze CS:GO demos in C#*. URL: https://github.com/StatsHelix/demoinfo. (accessed: 10.dec 2024).

[22] Niklaus Schiess. *Investigate the actual suspects of Counter-Strike: Global Offensive Overwatch cases*. URL: https://github.com/takeshixx/csgo-overwatcher.

[23] SteamDB. *Counter-Strike 2 (App 730) Patches and Updates*. URL: https://steamdb.info/app/730/patchnotes/. (accessed: 3.dec 2024).

[24] SteamDB. *Counter-Strike 2 update for 16 February 2023*. URL: https://steamdb.info/patchnotes/10562092/. (accessed: 3.dec 2024).

[25] SteamDB. *Release Notes for 3/22/2023*. URL: https://steamdb.info/patchnotes/10832117/. (accessed: 3.dec 2024).

[26] Valve Developer Community. *Counter-Strike: Global Offensive Access Match History - Valve Developer Community*. URL: https://developer.valvesoftware.com/wiki/Counter-Strike:_Global_Offensive_Access_Match_History. (accessed: 14.dec 2024).

[27] akiver. *CS Demo Manager*. URL: https://cs-demo-manager.com/. (accessed: 6.dec 2024).

[28] akiver. *CLI — CS Demo Manager*. URL: https://cs-demo-manager.com/docs/cli. (accessed: 14.dec 2024).

[29] CSStats. *CS2 Stats Automatic stat and match tracking*. 2020. URL: https://csstats.gg/.

[30] FACEIT. *FACEIT Challenge your game!* URL: https://www.faceit.com/en.

[31] Mille Mei Zhen Loo & Gert Lužkov. *Pinkvinus/CS2-demo-scraper: Counter-Strike share code scraper*. 2024. URL: https://github.com/Pinkvinus/CS2-demo-scraper.

[32] Python Software Foundation. *Python Release Python 3.11.0*. URL: https://www.python.org/downloads/release/python-3110/. (accessed: 10.dec 2024).

[33] Cloudflare. *Connect, protect, and build everywhere*. URL: https://www.cloudflare.com/.

[34] VeNoMouS. *cloudscraper 1.2.71*. URL: https://pypi.org/project/cloudscraper/. (accessed: 7.dec 2024).

[35] Leonard Richardson. *Beautiful Soup Documentation*. URL: https://www.crummy.com/software/BeautifulSoup/bs4/doc/. (accessed: 7.dec 2024).

[36] Python Software Foundation. *collections — Container datatypes*. URL: https://docs.python.org/3/library/collections.html. (accessed: 7.dec 2024).

[37] Python Software Foundation. *csv — CSV File Reading and Writing*. URL: https://docs.python.org/3/library/csv.html. (accessed: 7.dec 2024).

[38] Python Software Foundation. *datetime — Basic date and time types*. URL: https://docs.python.org/3/library/datetime.html. (accessed: 7.dec 2024).

[39] Python Software Foundation. *math — Mathematical functions*. URL: https://docs.python.org/3/library/math.html. (accessed: 7.dec 2024).

[40] Python Software Foundation. *os — Miscellaneous operating system interfaces*. URL: https://docs.python.org/3/library/os.html. (accessed: 7.dec 2024).

[41] Pandas. *pandas documentation*. URL: https://pandas.pydata.org/docs/. (accessed: 7.dec 2024).

[42] pygame. *Pygame Front Page*. URL: https://www.pygame.org/docs/. (accessed: 7.dec 2024).

[43] Python Software Foundation. *time — Time access and conversions*. URL: https://docs.python.org/3/library/time.html. (accessed: 7.dec 2024).

[44] Python Software Foundation. *re — Regular expression operations*. URL: https://docs.python.org/3/library/re.html. (accessed: 7.dec 2024).

[45] 2Captcha. *Bypass Cloudflare captcha: How to automatically solve and bypass Turnstile*. URL: https://2captcha.com/p/cloudflare-turnstile. (accessed: 10.dec 2024).

[46] CapMonster Cloud. *Cloudflare CAPTCHA Solving service - CapMonster Cloud. How to bypass Cloudflare CAPTCHA?* URL: https://capmonster.cloud/en/l/turnstile. (accessed: 10.dec 2024).

[47] Capsolver. *Cloudflare Turnstile Captcha Solver*. URL: https://www.capsolver.com/products/cloudflare. (accessed: 10.dec 2024).

[48] NoCapthca Ai. *Best Cloudflare Solver*. URL: https://nocaptchaai.com/p/cloudflare. (accessed: 10.dec 2024).

[49] Zenrows. *Solve and Bypass Cloudflare Turnstile CAPTCHAs: Easy Way*. URL: https://www.zenrows.com/captcha/turnstile. (accessed: 10.dec 2024).

[50] Counter-Strike Wiki. *Headshot*. URL: https://counterstrike.fandom.com/wiki/Headshot.

[51] liquipedia. *Counter-Strike 2 Maps - Liquipedia Counter-Strike Wiki*. URL: https://liquipedia.net/counterstrike/Portal:Maps/CS2.

[52] Steam. URL: https://api.steampowered.com/ISteamApps/GetSDRConfig/v1/?appid=730.

[53] FACEIT. *Acti-Cheat - FACEIT.com*. URL: https://www.faceit.com/en/anti-cheat.

[54] Flaresolverr. *Flaresolverr/Flaresolverr: Proxy server to bypass Cloudflare protection*. URL: https://github.com/FlareSolverr/FlareSolverr. (accessed: 10.dec 2024).

A CS2 report pop-up

Below is a pop-up that appears when pressing report on a users profile within a CS2 game[4].

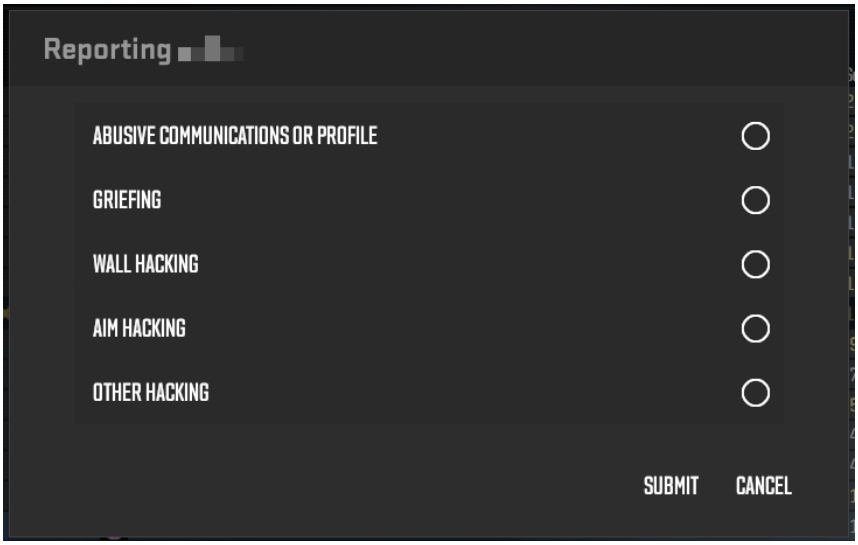


Figure 10: The popup shown when pressing report on a users account within Counter-Strike 2

B Server location data

Server name	Count
eu_west Server	2234
eu_north Server	2028
asia Server	972
poland Server	935
us_north_central Server	733
eu_east Server	701
australia Server	361
hong_kong Server	338
us_southwest Server	326
us_southeast Server	313
argentina Server	260
us_east Server	253
helsinki Server	248
spain Server	213
japan Server	208
india Server	208
south_america Server	205
london Server	191
india_east Server	77
us_west Server	60
alicloud_chengdu_ctuali Server	54
seoul Server	42
alicloud_shanghai_pvg Server	40
chile Server	33
pwrd_guangdong Server	24
pwrd_uc_tianjin Server	17
dubai Server	11
tencent_guangzhou_tgd Server	3

Table 4: Number of data points gathered from a specific server

The sum of matches played in EU would then be:

$$\begin{aligned} server_{EU} &= eu\_westServer + eu\_northServer + polandServer + eu\_eastServer \\ &\quad + helsinkiServer + spainServer + londonServer \\ &= 2234 + 2028 + 935 + 701 + 248 + 213 + 191 \\ &= 6803 \end{aligned}$$

Which as a percentage of the whole dataset would be:

$$percent_{EU} = \frac{6803}{11088} \approx 61.35\% \tag{1}$$

The sum of matches played in Asia would then be:

$$\begin{aligned} server_{Asia} &= asiaServer + hong\_kongServer + japanServer + indiaServer \\ &\quad + india\_eastServer + alicloud\_chengdu\_ctualiServer + seoulServer \\ &\quad + alicloud\_shanghai\_pvgServer + pwrd\_guangdongServer \\ &\quad + pwrd\_uc\_tianjinServer + tencent\_guangzhou\_tgdServer \\ &= 972 + 338 + 208 + 208 + 77 + 54 + 42 + 40 + 24 + 17 + 3 \\ &= 1983 \end{aligned}$$

Which as a percentage of the whole dataset would be:

$$percent_{Asia} = \frac{1983}{11088} \approx 17.88\% \tag{2}$$

The sum of matches played in America would then be:

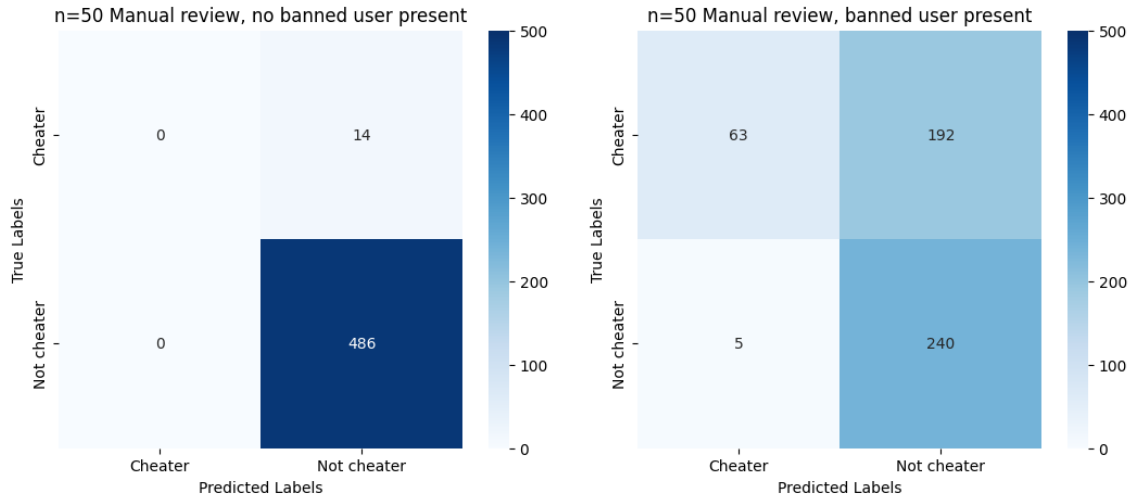
$$\begin{aligned} server_{America} &= us\_north\_centralServer + us\_southwestServer + us\_southeastServer \\ &\quad + argentinaServer + us\_eastServer + south\_americaServer \\ &\quad + us\_westServer + chileServer \\ &= 733 + 326 + 313 + 260 + 253 + 205 + 60 + 33 \\ &= 2183 \end{aligned}$$

Which as a percentage of the whole dataset would be:

$$percent_{America} = \frac{2183}{11088} \approx 19.69\% \tag{3}$$

C Calculation of precision and recall values

The following appendix entry are the calculations made for the precision and recall values for the manual review data sets as described in section 4.2.



(a) [$n = 50$ no VAC-banned user(s) present], The confusion matrix resulting from a manual review of 50 demo files with where no VAC-banned user was present

(b) [$n = 50$ VAC-banned user(s) present], The confusion matrix resulting from a manual review of 50 demo files where at least one VAC-banned user was present

Figure 11: Confusion matrices resulting from the manual review

C.1 Dataset with no banned users

Due to the fact that there are no predicted cheaters in this set, there will only be two calculation done as all other values are 0

$$\text{precision}_{\text{non cheater}} = \frac{486}{(486+14)} = \frac{243}{250} \approx 0.972 \tag{4}$$

$$\text{recall}_{\text{non cheater}} = \frac{486}{(486+0)} = 1 \tag{5}$$

C.2 Dataset with banned users

$$\text{precision}_{\text{non cheater}} = \frac{240}{(240+192)} = \frac{5}{9} \approx 0.556 \tag{6}$$

$$\text{recall}_{\text{non cheater}} = \frac{240}{(240+5)} = \frac{48}{49} \approx 0.98 \tag{7}$$

$$\text{precision}_{\text{cheater}} = \frac{63}{(63+5)} = \frac{63}{68} \approx 0.926 \tag{8}$$

$$\text{recall}_{\text{cheater}} = \frac{63}{(63+192)} = \frac{21}{85} \approx 0.247 \tag{9}$$